

✓ Two sum

Question

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

- Input: `nums = [2,7,11,15]`, `target = 9`
 - Output: `[0,1]`
 - Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.
-

Constraints

- $2 \leq \text{nums.length} \leq 10^4$
 - $-10^9 \leq \text{nums}[i] \leq 10^9$
 - $-10^9 \leq \text{target} \leq 10^9$
 - Only one valid answer exists.
-

Solution

so for this question here i do it by using hashmap..

analogy

- here i completed this question in $O(N)$ time complexity by using an `HashMap`.
- the idea is that we store number as key and index as values.
- in a single for loop we check the required number we need to get to the target as by applying `target - nums[i]`.
- then we ask our map that whether it contains that number or not.
- if yes then we return the array of index as we do in brute force approach
- if no then we put `nums[i]` as key and index as its value..
- when the loop is complete we get our desired result.

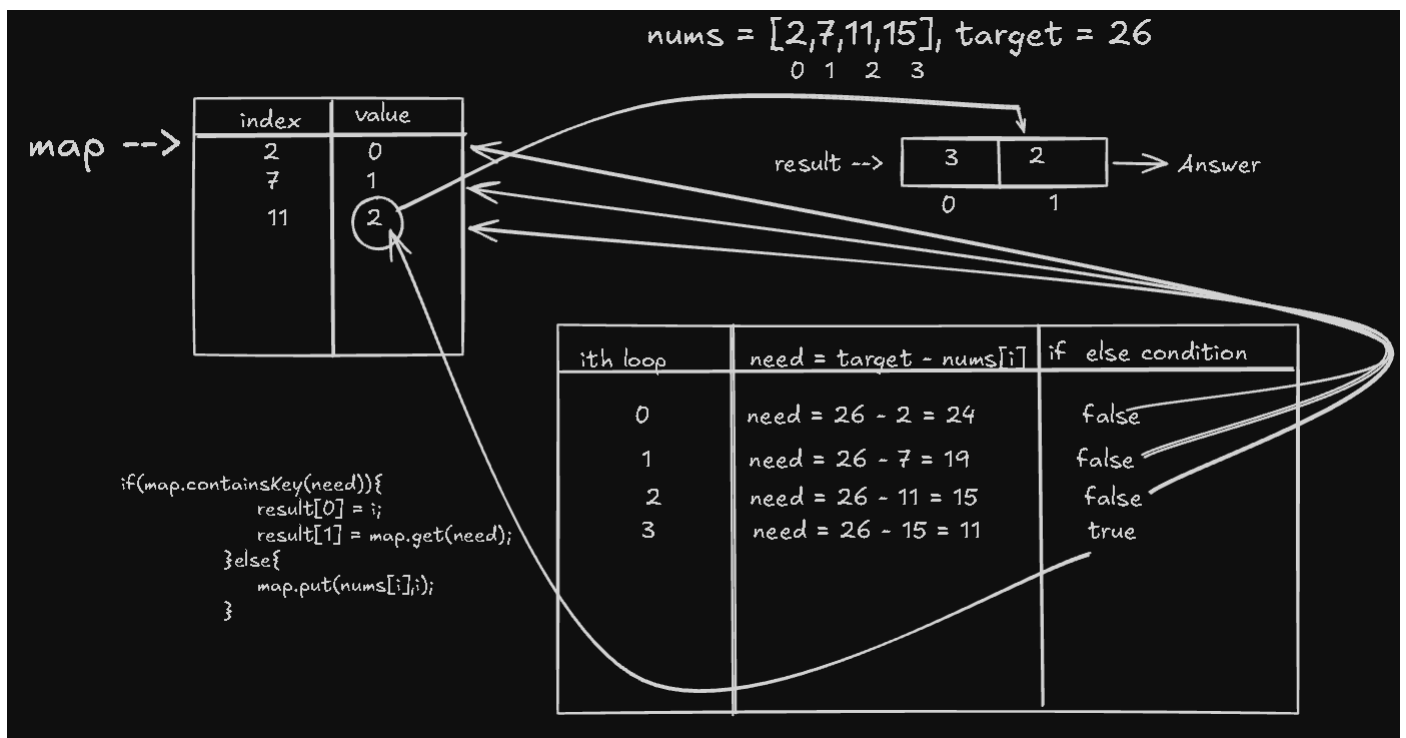
Code

```

class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap < Integer, Integer > map = new HashMap<>();
        int [] result = new int[2];
        for(int i = 0; i <= nums.length-1; i++){
            int need = target - nums[i];
            if(map.containsKey(need)){
                result[0] = i;
                result[1] = map.get(need);
            }else{
                map.put(nums[i],i);
            }
        }
        return result;
    }
}

```

✓ dry run of above code



time complexity = $O(n)$

space complexity = $O(n)$

